Estrutura de dados - ADS

Conteúdo da aula anterior ?

Pilha

Centro Paula Souza

Revisando aula 11

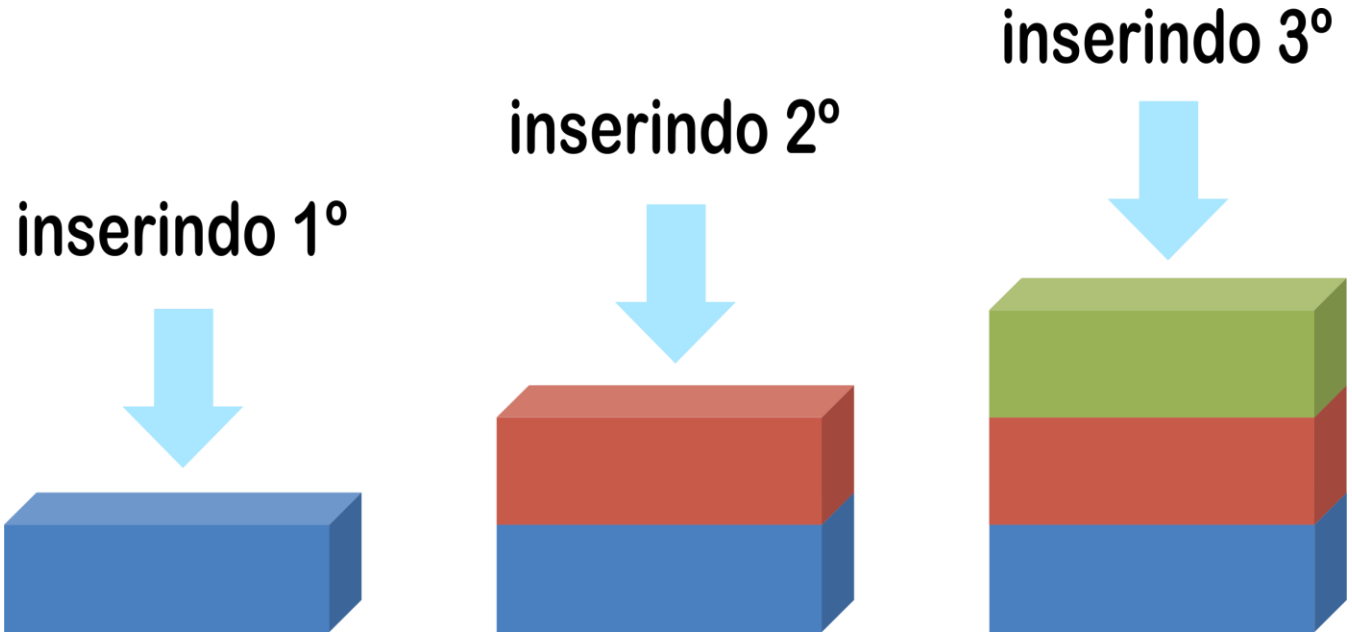
Pilhas - Stack

São estruturas de dados do tipo **LIFO** (last-in first-out), onde o último elemento a ser inserido, será o primeiro a ser retirado.

Assim, uma pilha permite acesso a apenas um item de dados - o último inserido. Para processar o penúltimo item inserido, deve-se remover o último.

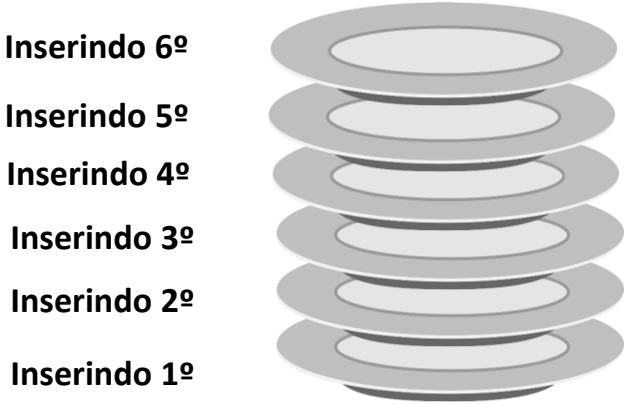
Pilhas - Stack

Empilhar - push



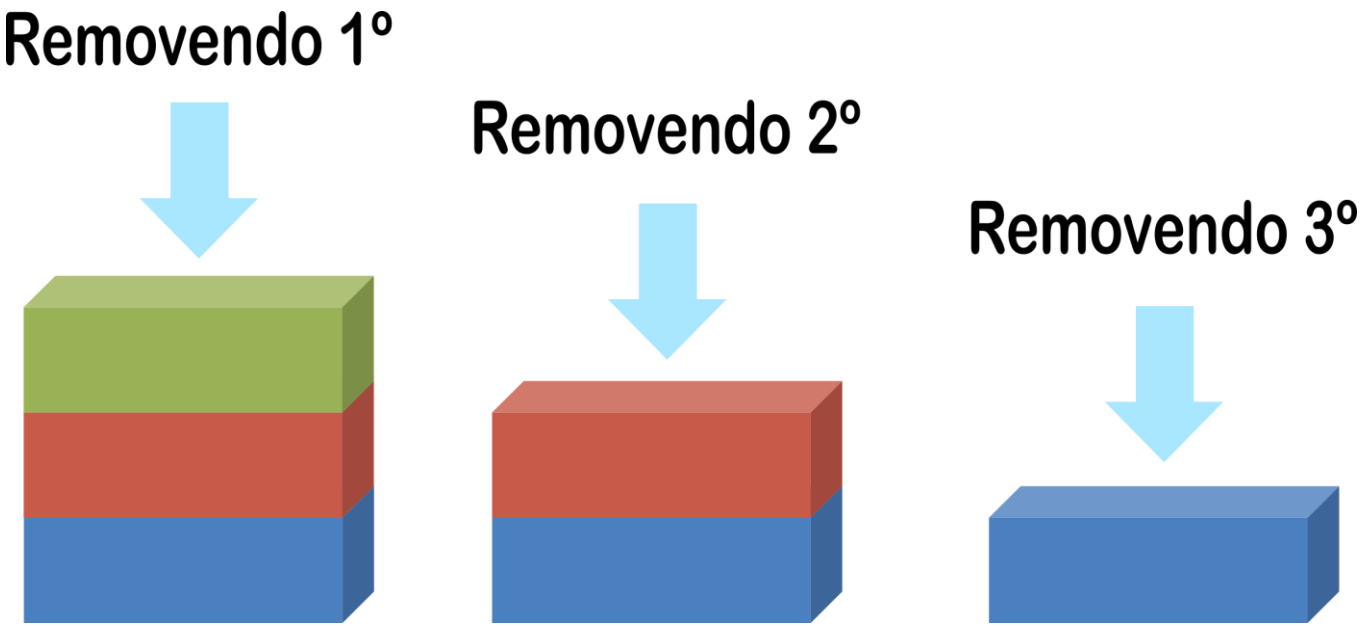
Pilhas - Stack

Empilhar - push



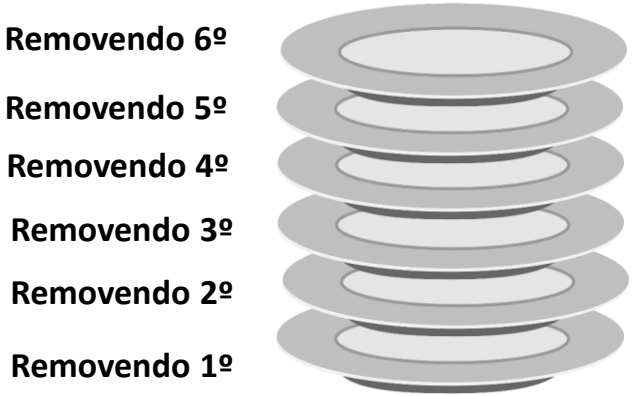
Pilhas - Stack

Desempilhar - pop



Pilhas - Stack

Desempilhar - pop



Pilhas - Stack

A implementação de pilhas pode ser realizada através de vetor (alocação do espaço de memória para os elementos é imediata) ,através de listas **ou stack** .

Numa pilha, a manipulação dos elementos é realizada em apenas uma das extremidades, chamada de topo, em oposição a outra extremidade, chamada de base.

Operações com Pilha

Todas as operações em uma pilha podem ser imaginadas como as que ocorre numa pilha de pratos em um restaurante ou como num jogo com as cartas de um baralho:

√Criação da pilha

√Empilhar (push)

O elemento é o parâmetro nesta operação;

√Desempilhar (pop);

Sempre o elemento do topo da lista

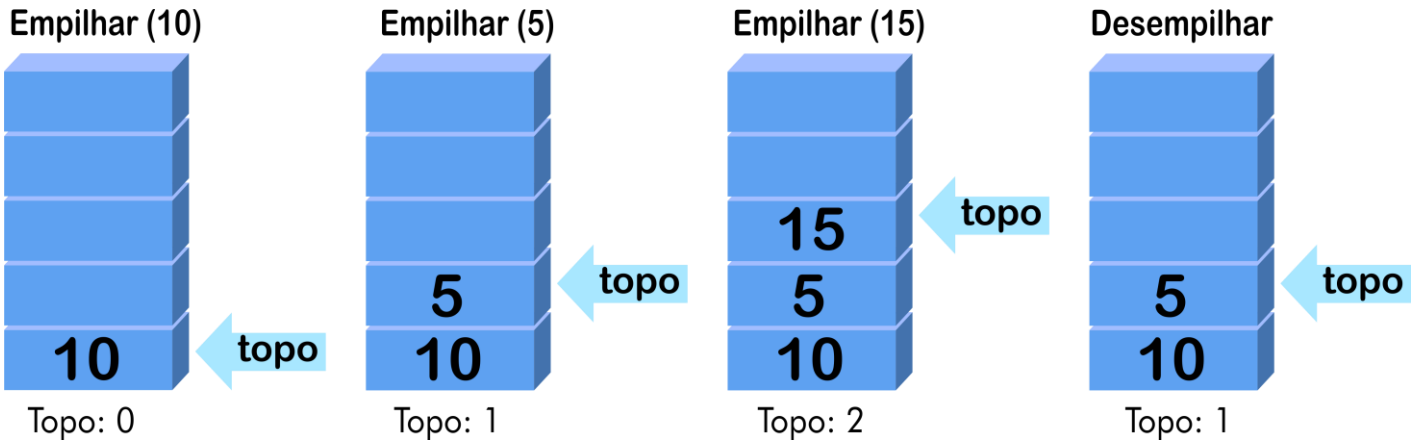
√Mostrar o topo;

√Verificar se a pilha está vazia

√Verificar se a pilha está cheia

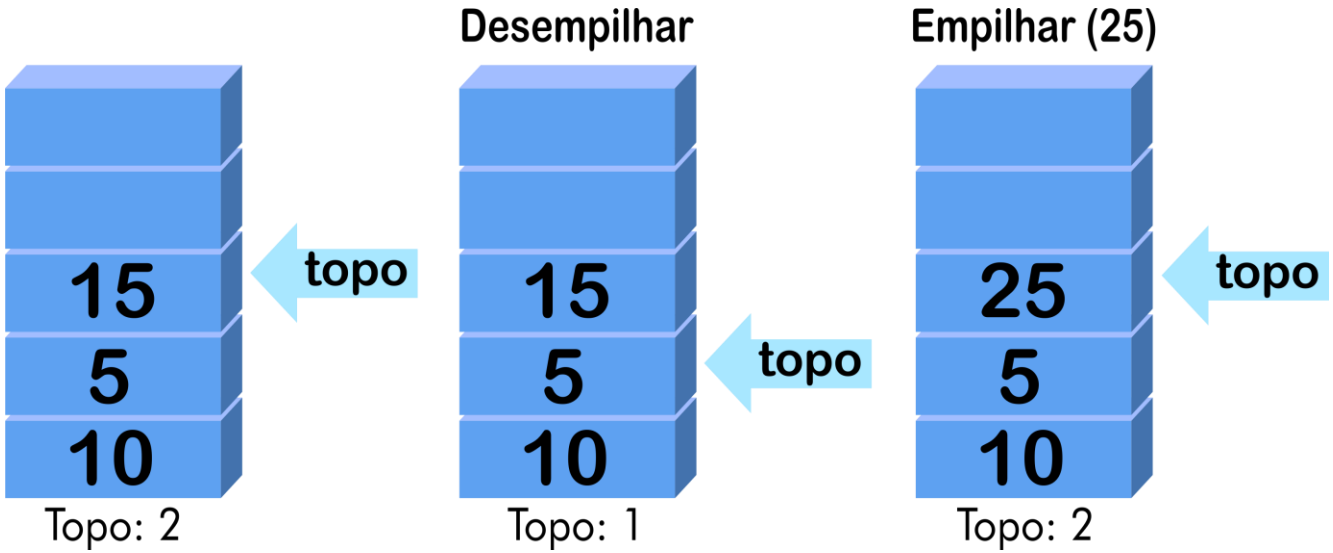
Pilhas - Stack

Supondo uma pilha com capacidade para 5 elementos (5 nós)



Pilhas - Stack

Na realidade a remoção de um elemento da pilha é realizada apenas alterando-se a informação da posição do topo.



Pilhas - Stack

Exemplo:

```
#include <iostream>
#include <stack>

using namespace std;

int main() {
  stack<int> pilha;
  pilha.push(1);
  cout <<pilha.top()  << endl;
  pilha.push(3);
  cout <<pilha.top()  << endl;
  pilha.push(5);
  cout <<pilha.top()  << endl;
  pilha.pop();

  // Mostra o valor no topo da pilha
  cout << pilha.top() << endl;

  return 0;
}
```

Pilhas - Stack

```
#include <iostream>
#include <stack>

using namespace std;

int main() {
  string expressao = "();";

  stack<char> pilha;
  for (int i = 0; i < expressao.length(); i++) {
    char caracter = expressao[i];

    if (caracter == '(') {
      pilha.push(caracter);
    }
    if (caracter == ')' ) {
      pilha.push(caracter);
    }
    else if (caracter == ';') {
      if (pilha.empty() || pilha.top() != caracter) {
        cout << "FIM"<< endl;
        return 0;
      }
      pilha.pop();
    }
  }

  if (pilha.empty()) {
    cout << "Pilha Vazia" << endl;
  }

  return 0;
}
```

```cpp
#include <iostream>
#include <stack>

using namespace std;

int main() {
  string expressao = "();";

  stack<char> pilha;
  for (int i = 0; i < expressao.length(); i++) {
    char caracter = expressao[i];

    if (caracter == '(') {
      pilha.push(caracter);
    }
    if (caracter == ')' ) {
      pilha.push(caracter);
    }
    else if (caracter == ';') {
      if (pilha.empty() || pilha.top() != caracter) {
        cout << "FIM"<< endl;
        // Desemplihamento
        while (!pilha.empty()){
            pilha.pop();
        }
        return 0;
      }
    }
  }

  if (pilha.empty()) {
    cout << "Pilha Vazia" << endl;
  }

  return 0;
}
```

Correção:

Agenda

Aula de hoje

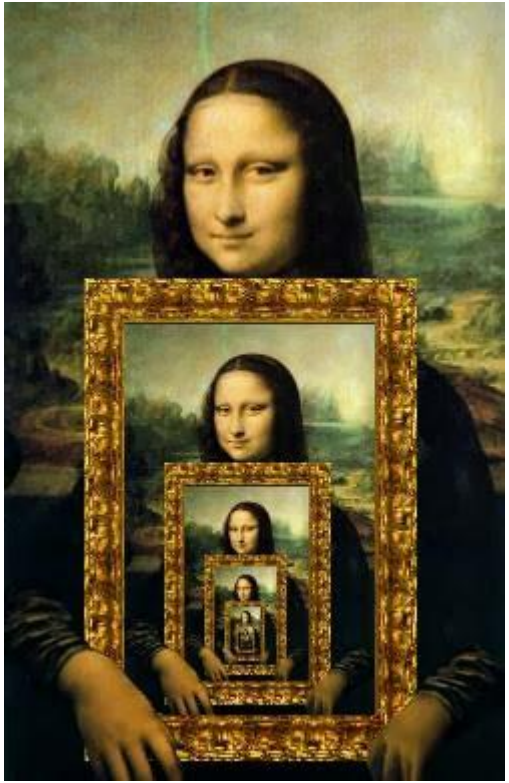
- Recursão

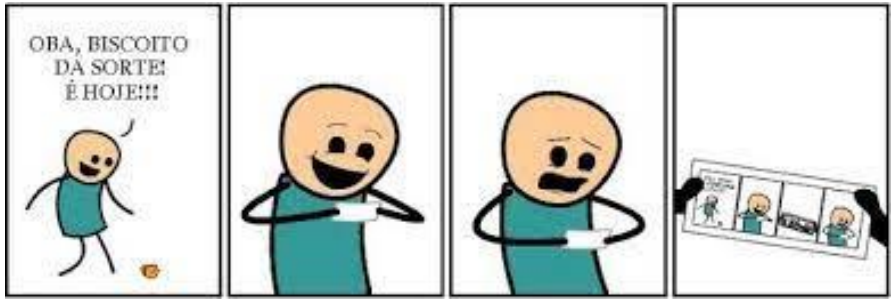
AGENDA

Centro Paula Souza

Recursão

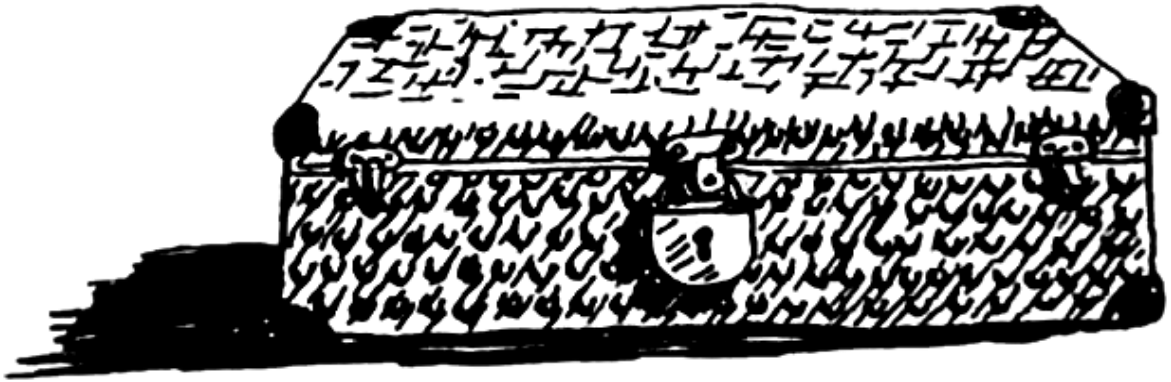
É quando uma função chama ela mesma...





Recursão

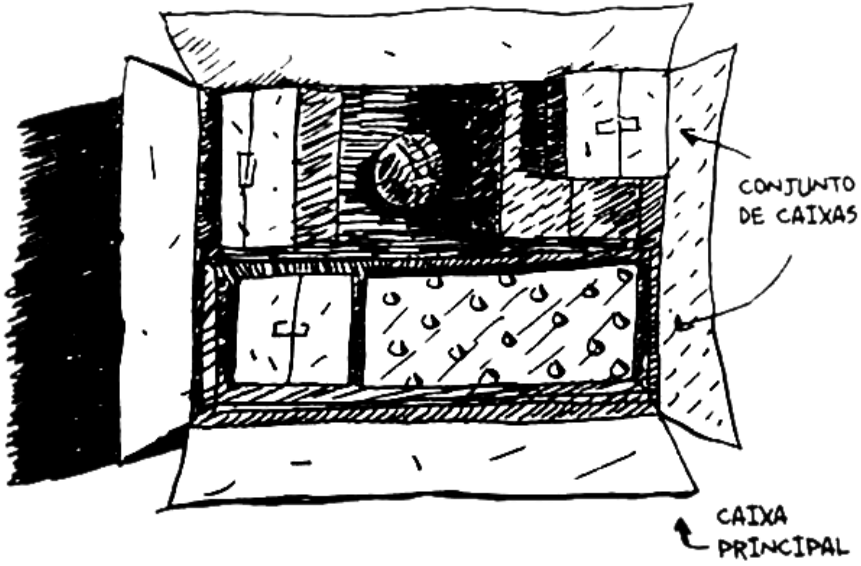
Suponha que você esteja explorando um armário e encontre uma misteriosa mala trancada



Recursão

A sua mãe diz que a chave da mala provavelmente está em uma caixa.

Esta caixa contém mais caixas com mais caixas dentro delas.
A chave está em alguma destas caixas.



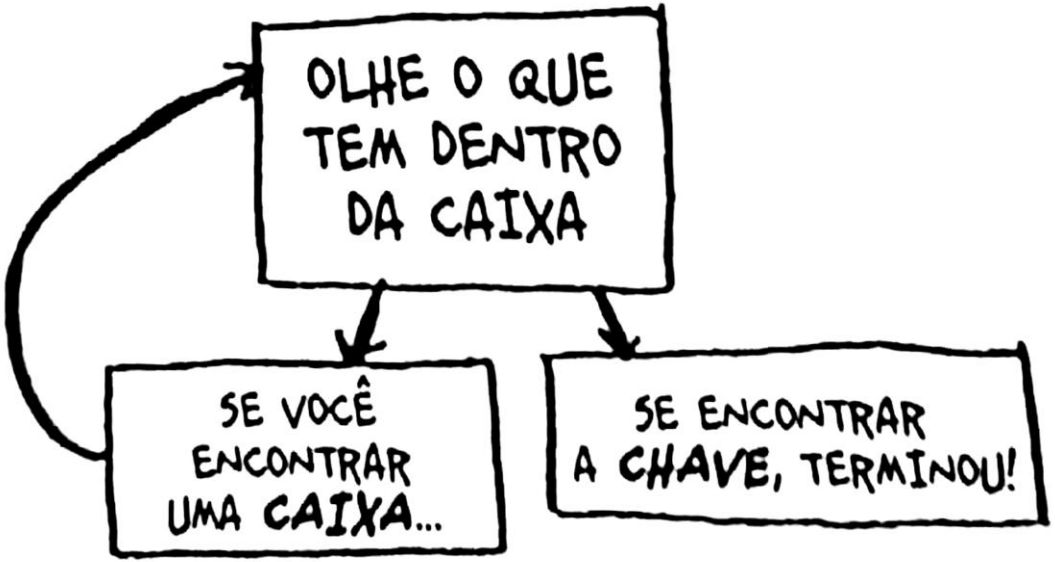
Qual é o seu algoritmo para procura-la?

Recursão

Abordagem 1

Recursão

Abordagem 2



Caso-base e caso recursivo

Quando se escreve uma função recursiva, **deve-se informar quando a recursão deve parar**. Para atender essa premissa toda função recursiva possui duas partes:

Caso Base

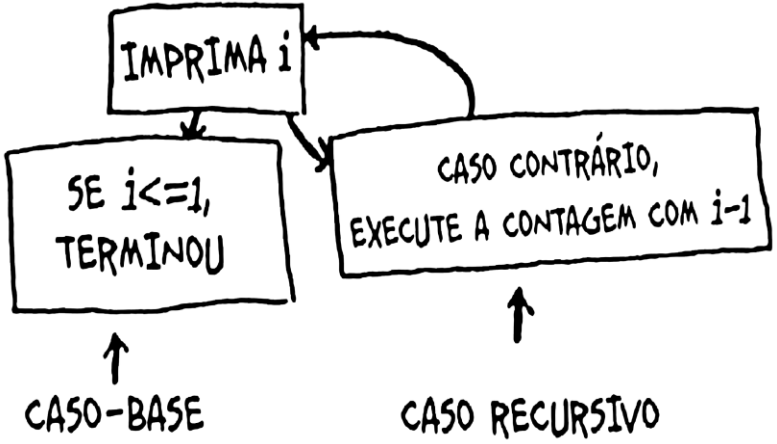
É o caso mais simples. É usada uma condição em que se resolve o problema com facilidade.

Caso Recursivo

Procuram simplificar o problema de tal forma que convergem para o caso base.

Caso-base e caso recursivo

Exemplo: Contagem Regressiva




```
#include <iostream>
using namespace std;
void regressiva(int i) {
  cout << i << endl;
  if (i <= 0) {
    return;
  }
  else {
    regressiva(i - 1);
  }
}

int main() {
  regressiva(10);
  return 0;
}
```

Pilha de Chamada (Call Stack)

Você tem uma lista de afazeres em forma de uma pilha de notas adesivas.

Quando você insere um item, ele é colocado no **topo da pilha (PUSH)**. Quando você lê um item, lê apenas o item do topo da pilha e **ele é retirado da lista (POP)**.

Logo, sua lista de afazeres contém apenas duas ações:

PUSH
(ADICIONE UM NOVO ITEM AO TOPO)

POP
(REMOVA O ITEM DO TOPO E LEIA-O)

Pilha de Chamada (Call Stack)

O Sistema Operacional usa uma pilha interna denominada **pilha** de chamada.

Um exemplo simples para ilustrar:

```
#include <iostream>
using namespace std;

void imprimir_sequencia(int n) {
  if (n == 0) {
    return;
  } else {
    cout << n << endl;
    imprimir_sequencia(n - 1);
  }
}

int main() {
  imprimir_sequencia(5);
  return 0;
}
```

Pilha de Chamada (Call Stack)

Suponha que você chame sauda("Maria"). Primeiro, seu SO aloca um espaço de memória para essa chamada.

Agora, vamos usar a memória. A variável nome é setada para "Maria". Isso precisa ser salvo.

"

Pilha de Chamada (Call Stack)

Cada vez que você faz uma chamada de função,o SO salva na memória os valores para todas as variáveis.

Depois disso, imprime olá, Maria!. Então, chama sauda2("Maria").

Novamente, o SO aloca uma caixa de memória para essa chamada de função.

Repetição x Recursão

Demonstração Call Stack

Repetição x Recursão

```
#include <iostream>
using namespace std;

int main() {
  for (int i = 0; i < 5; i++) {
    cout << "recursão" << endl;
  }
  return 0;
}
```

Saída

```
Recursão
Recursão
Recursão
Recursão
Recursão
```

X

```
#include <iostream>
using namespace std;
void imprimir_recursivamente(int n) {
  if (n == 0) {
    return;
  }            #caso base
  else {
    cout << "recursão" << endl;
    imprimir_recursivamente(n - 1);#caso recursivo
  }
}

int main() {
  imprimir_recursivamente(5);
  return 0;
}
```

Saída

```
Recursão
Recursão
Recursão
Recursão
Recursão
```

Repetição x Recursão

```cpp
#include <iostream>
using namespace std;
int soma_for(int n) {
  int soma = 0;
  for (int i = 0; i <= n; i++) {
    soma += i;
  }
  return soma;
}

int main() {
  cout << soma_for(5) << endl;
  return 0;
}
```

Saída

15

X

```cpp
#include <iostream>
using namespace std;

int soma_recursiva(int n) {
  if (n == 0) {
    return 0;
  }
  else {        #caso base
    return n + soma_recursiva(n - 1);
  }                                  #caso recursivo
}

int main() {
  cout << soma_recursiva(5) << endl;
  return 0;
}
```

Saída

15

Repetição x Recursão

Demonstração Call Stack

Desafios aula de hoje

Atividade

Crie uma função recursiva que implemente fatorial de um número.

Ex: 15!

Atividade

Crie uma **função recursiva** para verificar o valor que está na posição 9 da sequência de Fibonacci.

Atividade

Crie uma **função recursiva** que calcule o valor de a (base) elevado a b (expoente).

Se o expoente for zero, a potência é igual 1 (critério de parada)

Obs. Não considere exponenciação de números negativos

Até mais pessoal!